Dominant Strings for 3-SAT: Counting Solutions with Few Occurrences per Variable

Siyuan Zhuang

November 9, 2026

Abstract

Let $\Delta \geq 1$ and $\varepsilon > 0$. We count the solutions of a formula with clauses of at most three literals on n variables, in which every variable occurs at most Δ times, in time $Cn^c 2^{\varepsilon n}$, where C and c depend only on Δ and ε. Together with the sparsification lemma of Impagliazzo, Paturi and Zane, this refutes the Exponential Time Hypothesis, a statement first announced by OpenAI and Claude (Anthropic).

The proof is a recursion on a fixed tower of grids. A formula becomes a network of nodes joined by wires, and each wire is marked by the difference of the bits at its two ends. With this mark, two wires that cross both the rows and the columns of a grid satisfy $X = V - H$: the crossing pair equals a vertical pair minus a horizontal pair. For t such pairs on one rectangle, the product splits into 2^t strings over $\{V, H\}$. We sum the strings with more letters H across the rows and the strings with more letters V across the columns. Equal product terms then coincide, and each half is carried by at most $1.8t$ new wires in place of $2t$. A quadratic potential falls by the factor $\sqrt{0.91}$ at every level in one of the two directions, whatever the distribution of the wires. The proof uses one identity on four bits, a count of strings and the Cauchy–Schwarz inequality. The theorem is formalized in Lean 4 down to a multitape machine, with its time and space.

The result and the strategy

An *occurrence* of a variable i is a pair of a clause C and a literal x_i or $\neg x_i$ in C.

Theorem 1. *Let $\Delta \geq 1$ and $\varepsilon > 0$. Given a formula Ψ on the variables $x_1, \ldots, x_n$ with clauses of at most three literals, in which every variable has at most Δ occurrences, the number of assignments of the n variables that satisfy Ψ is computed in time $Cn^c 2^{\varepsilon n}$, where C and c depend only on Δ and ε.*

The Exponential Time Hypothesis of Impagliazzo and Paturi [IP01] says that for some $\delta > 0$, no algorithm decides 3-SAT on n variables in time $2^{\delta n}$ times a polynomial in the input size. The sparsification lemma of Impagliazzo, Paturi and Zane [IPZ01] reduces every 3-CNF to formulas with few occurrences, so Theorem 1 contradicts the hypothesis:

Corollary 2. *For each $\varepsilon > 0$, 3-SAT with n variables is decided deterministically in time $C_\varepsilon |\varphi|^{O_\varepsilon(1)} 2^{\varepsilon n}$, where $|\varphi|$ is the size of the input φ.*

Background and what is simplified

OpenAI [OAI26] and, independently, Claude (Anthropic) [Cla26] announced algorithms for counting with explicit subexponential rates. O'Donnell [O'D26] and Kintali [Kin26] gave expositions of OpenAI's approach, and a community examination of the argument has been called for [ETH26]. Like the proof of Theorem 2 in [Cla26] and like Kintali, we fix ε and leave the rate unoptimized. We aim for a proof that is short to state and easy to follow.

The proof was written to be presented in an explanatory video and to a general audience, so its steps are elementary and can be drawn. For the same reason it uses plain names for standard objects. In the language of tensor networks, a network is a tensor network: a node function is a tensor, the w wires between two nodes form one bond of dimension 2^w, the value Z is the full contraction, and merging the nodes of a row contracts them. The mark of a wire is the 2×2 Levi-Civita matrix, $\epsilon_{e^- e^+} = e^+ - e^-$, and an equality test is the identity matrix; Lemma 3 changes one into the other. The identity $X = V - H$ is the Schouten identity $\epsilon_{xv}\epsilon_{yu} = \epsilon_{xu}\epsilon_{yv} - \epsilon_{xy}\epsilon_{uv}$, and the strings and letters of Section 3 index the terms of a multiplied-out product.

The closest earlier route is Claude's. The grid of rows and columns, the rectangles, the expansion with one piece per rectangle, and the choice to merge rows or columns for each term are the core of Claude's recursion (Sections 2.5, 4 and 5 of [Cla26]). The identity $X = V - H$ for one pair of wires is the one-bit crossing identity of [Cla26, Appendix H], written with difference marks; it is also the classical relation among 2×2 determinants. Two parts differ from Claude's proof. First, the pieces of a rectangle: the proof of Theorem 2 in [Cla26] applies an eight-square identity to blocks of three bits, with a number of pieces per rectangle that grows with the bits, and it regroups the merged nodes by a second-moment argument (Section 2.6 and Lemma 1 there). Here each rectangle has exactly two pieces, the sums over the H-dominant and the V-dominant strings, and the count of their distinct product terms (Lemma 5) gives at most $1.8t$ wires for t pairs. Second, what is tracked: a quadratic potential falls at every level for arbitrary weights (Lemma 7), so no node is regrouped, and the number of nodes at each level is the square root of the last one. In return, Claude's route gives an explicit rate: with pieces built from integer partitions the potential falls by $1/\sqrt{2}$ per level (Appendices A and B of [Cla26]), where ours falls by $\sqrt{0.91} \approx 0.954$.

Both differences are shared with the companion paper *Uncrossing Wires for 3-SAT* [Zhu26], which introduced the quadratic potential tracked across the levels and the two pieces grouped by the number of horizontal terms. The present paper reads independently of it and changes three things. First, the mark of a wire. In [Zhu26] a wire tests whether its two end bits are equal, and the crossing identity reads $[x = v][y = u] = U - S$, where $U = [x = u][y = v]$ is again a pair of equality tests and $S = (y - x)(v - u)$ is a product of differences. Here every wire carries the difference of its end bits, so all three terms of $X = V - H$ are products of two differences, and every level after the first is a network of this one kind (Lemma 3 converts the equality tests of the formula). The four letters P, Q, R, H of Section 3 take the place of the four functions of two bits in [Zhu26]. Second, the number of pairs used on a rectangle. In [Zhu26] it is the largest odd number at most the smaller diagonal weight w_2, because with an even number m the index there saves nothing at $m = 2$. Here a tie between H and V is broken by the last letter, so the index has $t - 1 + \lceil 8\lfloor t/2 \rfloor / 5 \rceil$ bits, all $t = w_2$ pairs are used, and the cheaper piece costs at most $w_1 + 0.8w_2$ wires instead of $w_1 + 0.9w_2$. The potential then falls by $\sqrt{0.91}$ per level instead of $\sqrt{24/25}$. Third, the statement. The theorem of [Zhu26] is the 3-SAT theorem, with the branching of Impagliazzo, Paturi and Zane inside its proof. Theorem 1 counts the solutions of formulas with few occurrences and has no branching, and its Lean form `Machine.machine_bounded` does not

use the branching tree (Section 8). Quadratic quantities also occur in the other cited works as balance tools inside one level or one schedule: the variance in Claude's Lemma 1, and the potentials of Kintali's schedules.

The strategy

A sum over assignments would split if the clauses respected some partition of the variables into two groups: then it is a product of two sums. Every wire that crosses the partition multiplies the number of products by two. A formula with few occurrences can be an expander, where every balanced partition is crossed by a constant fraction of the occurrences. So we do not look for one partition.

We write the number of solutions as the value of a network: nodes joined by wires, where each wire is marked by the difference of the bits at its two ends (Section 1). Then we arrange the nodes in a grid and use two partitions at once, into rows and into columns. Two wires on the diagonals of a rectangle cross both. The identity $X = V - H$ replaces them by a pair that crosses only the rows and a pair that crosses only the columns (Section 2). A rectangle with t pairs gives 2^t strings over $\{V, H\}$; we group them into two pieces, sum each piece across the cheaper cut, and merge equal product terms, so that each piece costs at most $1.8t$ wires across its cut instead of $2t$ (Section 3). Choosing one piece per rectangle gives a sum of terms whose number depends only on the grid. Each term is a network on the rows or on the columns, after merging the nodes of a row or of a column (Section 4). A term may mix the pieces of different rectangles, so the number of wires need not fall, but a quadratic potential falls by $\sqrt{0.91}$ in one of the two networks (Section 5). We keep that network and repeat on a tower of grids, until two nodes remain (Section 6). The potential is $O(\Delta n)$ when the variables and clauses are spread evenly over the nodes, and so the exponent becomes εn (Section 7).

Throughout, a statement in square brackets stands for 1 if it holds and 0 otherwise, and $[K] = \{0, \dots, K-1\}$. *Cauchy* means $(\sum_{i\in I} c_i d_i)^2 \le \sum_{i\in I} c_i^2 \sum_{i\in I} d_i^2$; with all $d_i = 1$ it reads $(\sum_{i\in I} c_i)^2 \le |I| \sum_{i\in I} c_i^2$.

1 Networks

Definition. A *network* has *nodes* and *wires*. Each wire e joins two different nodes, its *tail* and its *head*, and it has two *ends*, one at each of these nodes. A *configuration* gives a bit to every end. We write e^- for the bit at the tail end of e and e^+ for the bit at its head end. The *mark* of e is the number $e^+ - e^- \in \{-1, 0, 1\}$. Every node s has a *node function* F_s: it reads the bits of the ends at s and returns a dyadic rational, that is, a fraction with a power of 2 as denominator. The *value* of the network N is

$$Z(N) = \sum_{\text{configurations}} \prod_{\text{wires } e} (e^+ - e^-) \prod_{\text{nodes } s} F_s.$$

The mark of e is 0 unless $e^+ = 1 - e^-$, and then it is $(-1)^{e^-}$. So only the configurations in which every head end holds the complement of its tail end contribute, and there are $2^{|E|}$ of them, where $|E|$ is the number of wires. Enumerating them costs $2^{|E|}$ evaluations of the node functions. Our aim is to write Z as a short sum of values of networks with few wires. The *weight* $w(s,t) = w(t,s)$ is the number of wires between the nodes s and t, in either orientation; $w(s,s) = 0$.

Reversing a wire, that is, exchanging its tail and its head, negates its mark. So reversing a wire and multiplying the function of one of its two nodes by -1 keeps the value. We use this to orient wires as we like.

Equality tests. The same nodes, wires and node functions also define a value in which each wire carries one bit: the value with *equality tests* is

$$Z_{=}(N) = \sum_{\text{configurations}} \prod_{\text{wires } e} [e^+ = e^-] \prod_{\text{nodes } s} F_s.$$

A formula is easy to write in this form. The difference marks come from it by the following conversion: complement every head end, and give every tail end the sign $(-1)^{e^-}$.

Lemma 3. *Let N be a network, and let N' have the same nodes and wires and the node functions*

$$F'_s = \prod_{\textit{tail ends at } s} (-1)^{e^-} \cdot F_s(\textit{the ends at } s, \textit{ each head end } e^+ \textit{ replaced by } 1 - e^+).$$

Then $Z(N') = Z_{=}(N)$.

Proof. In $Z(N')$ only the configurations with $e^+ = 1 - e^-$ for every wire contribute, and the mark of e is then $(-1)^{e^-}$. The node function at the head of e reads $1 - e^+ = e^-$, and the node function at its tail has the sign $(-1)^{e^-}$, which cancels the mark. So each contributing configuration of N' gives the term of $Z_{=}(N)$ in which both ends of every wire hold the bit of its tail end, and each term of $Z_{=}(N)$ arises once. □

The network of a formula. Let Ψ be a formula on $x_1, \dots, x_n$ whose clauses are sets of literals. Its network has a node for each variable i, a node for each clause C, and one more node for each variable, its *own* node. It has a wire for each occurrence, from the variable to the clause, and a wire from each variable to its own node. With equality tests each wire holds one bit. The node function of a variable is 1 if all bits of its wires are equal and 0 otherwise; the node function of a clause is 1 if some literal of C is true when the literal x_i reads the bit b of its wire and $\neg x_i$ reads $1 - b$, and 0 otherwise; the node function of an own node is 1.

Lemma 4. *$Z_{=}$ of the network of Ψ is the number of assignments of $x_1, \dots, x_n$ that satisfy Ψ.*

Proof. A configuration with a nonzero term gives all wires of the variable i one common bit x_i; every variable has at least its own wire, so the configurations with nonzero terms correspond to the assignments $x \in \{0,1\}^n$. The term of x is the product of the clause functions, which is 1 if every clause has a true literal and 0 otherwise. □

Example. For the single clause $x_1 \vee \neg x_2$, Lemma 3 gives the difference form. All wires of a variable have their tails at the variable, so its function becomes $(-1)^{s_1+\dots+s_{k+1}}[s_1 = \dots = s_{k+1}]$ on its $k+1$ wires, that is [all 0] + $(-1)^{k+1}$[all 1]. The clause has only head ends, and its function becomes $1 - l_1(1 - l_2)$, because the literal x_1 is false when the complemented bit $1 - l_1$ is 0. An own node keeps the function 1. Summing over the contributing configurations gives 3, the number of solutions.

Merging. Let the nodes be partitioned into *blocks*. The *merged network* has a node for each block β, the wires between different blocks with their orientations, and the node function

$$F_\beta = \sum \prod_{e \text{ inside } \beta} (e^+ - e^-) \prod_{s \in \beta} F_s, \tag{1}$$

where the sum runs over the bits of the ends of the wires inside β, and the ends of the other wires at nodes of β are the inputs of F_β. Multiplying out $\prod_\beta F_\beta$ and reordering the sums shows that the merged network has the same value. One evaluation of F_β is a sum of at most 2^b terms, where b is the number of ends at nodes of β, and each term has one evaluation of F_s for each $s \in \beta$. With equality tests the same holds, with one bit for each wire inside β and the test $[e^+ = e^-]$ in place of the mark.

2 Two cuts, and the crossing of two wires

Let the nodes be split into two sets L and R. If no wire joins L to R, the value is a product: $Z(N) = Z(N_L)Z(N_R)$, where N_L and N_R keep the nodes and wires on each side. If s wires join them, fix the bits of their ends: the contributing configurations of these wires are 2^s, so $Z(N)$ is a sum of 2^s products of a number that depends only on L and a number that depends only on R. We call these numbers the *product terms* of the cut. So s crossing wires cost a factor 2^s, and conversely a sum of 2^s product terms across a cut can be carried by s wires (Section 4).

We use two cuts at once. Arrange the nodes in a grid of rows and columns. The *row cut* separates two rows and the *column cut* two columns. A wire inside a row crosses only the column cut, and a wire inside a column only the row cut. A wire between different rows and different columns crosses both.

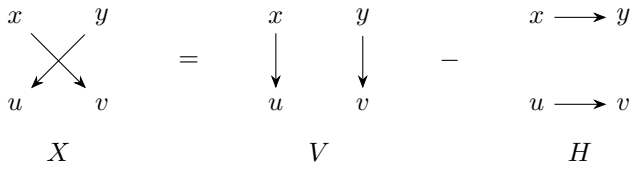
Take two rows and two columns. Their four nodes form a *rectangle*: top left, top right, bottom left, bottom right. Let one wire run from the top left to the bottom right, and another from the top right to the bottom left, both with the tail in the top row. Write x and y for their tail ends, at the top left and the top right, and v and u for their head ends, at the bottom right and the bottom left. The product of their marks is the *crossing* $X = (v - x)(u - y)$. Put

$$V = (u - x)(v - y), \qquad H = (x - y)(u - v).$$

These are the products of the marks of a *vertical* pair, from x down to u and from y down to v, and of a *horizontal* pair, from x to y and from u to v, whose marks multiply to $(y-x)(v-u) = H$. Expanding the three products gives

$$X = V - H, \tag{2}$$

since both sides equal $uv - xu - yv + xy$. This is the relation among the 2×2 determinants of the columns $(1, x), (1, y), (1, u), (1, v)$: the mark $e^+ - e^-$ is the determinant of the columns $(1, e^-)$ and $(1, e^+)$.



Multiply (2) by the rest of the network and sum. Then the value with the crossing pair is the value with a vertical pair minus the value with a horizontal pair. The vertical pair crosses only the row cut, so across the column cut it costs no product term; the horizontal pair crosses only the column cut. One crossing pair costs $2^2 = 4$ product terms across either cut. After (2), the vertical network across the column cut and the horizontal network across the row cut cost one product term each, two in all.

The mark matters here. With the mark $[e^+ = e^-]$, which tests equality, no fixed numbers a, b satisfy $Z_X = aZ_V + bZ_H$ for all networks, where Z_X, Z_V, Z_H are the values with the crossing, vertical and horizontal pair. Indeed, let the rest of the network contribute $(1-x)\,y\,u\,(1-v)$, which is 1 only at $x=0$, $y=1$, $u=1$, $v=0$. With equality marks, the crossing pair joins $x=0$ to $v=0$ and $y=1$ to $u=1$, so $Z_X = 1$; the vertical pair joins $x=0$ to $u=1$, so $Z_V = 0$; the horizontal pair joins $x=0$ to $y=1$, so $Z_H = 0$.

3 Many pairs: the dominant strings

A rectangle can carry many crossing pairs. Let $t \geq 1$, and let the j-th pair have the ends x_j, y_j, u_j, v_j as above, for $1 \leq j \leq t$; write X_j, V_j, H_j for the products of this pair. By (2), $\prod_j X_j = \prod_j (V_j - H_j)$. A *string* is a word $\sigma = \sigma_1 \cdots \sigma_t$ over the letters V and H; σ_t is its *last letter*. Multiplying out,

$$\prod_{j=1}^{t} X_j = \sum_{\sigma} T_\sigma, \qquad T_\sigma = \prod_{j:\sigma_j = V} V_j \prod_{j:\sigma_j = H} (-H_j).$$

What a string costs. Across the row cut, $H_j = (x_j - y_j)(u_j - v_j)$ is one product term, of a function of the top ends and a function of the bottom ends. Also

$$V_j = x_j y_j \cdot 1 + 1 \cdot u_j v_j - \tfrac{1}{2}(x_j + y_j)(u_j + v_j) + \tfrac{1}{2}(x_j - y_j)(u_j - v_j), \tag{3}$$

since the right side is $x_j y_j + u_j v_j - x_j v_j - y_j u_j$. We name the four terms by the letters P, Q, R, H. So a string with d letters V is a sum of 4^d product terms across the row cut, and in the same way of 4^{t-d} across the column cut. Treating the 2^t strings one by one, with the cheaper cut for each, costs about 4^t product terms in all when t is large, because most strings have about $t/2$ letters of each kind. This saves no fixed fraction of the 4^t terms of the crossing.

Equal product terms. The last term of (3) is half of the product H_j itself. So if a string has the letter V at the position j and the expansion (3) takes the letter H there, the product term is the same as for the string with H at the position j. Equal product terms need to be computed once: we add their coefficients. This works within one cut, so we split the strings into two halves, one for each cut.

A string is *H-dominant* if it has fewer letters V than H, or as many and the last letter H; otherwise it is *V-dominant*. Let $\mathcal{H}$ and $\mathcal{V}$ be the two sets, and put

$$D_H = \sum_{\sigma \in \mathcal{H}} T_\sigma, \qquad D_V = \sum_{\sigma \in \mathcal{V}} T_\sigma, \qquad \prod_{j=1}^{t} X_j = D_H + D_V.$$

The *complement* $\bar{\sigma}$ exchanges the letters V and H at every position. Exactly one of $\sigma, \bar{\sigma}$ is H-dominant: if the numbers of the two letters differ, the complement exchanges them; if they are equal, the last letters of σ and $\bar{\sigma}$ differ. So $|\mathcal{H}| = |\mathcal{V}| = 2^{t-1}$. An H-dominant string has at most $\lfloor t/2 \rfloor$ letters V. Changing a letter V into H keeps a string H-dominant: if it had fewer V than H, it still has; if it had as many and the last letter H, it now has fewer V.

For $t = 3$, $\mathcal{H} = \{HHH, VHH, HVH, HHV\}$. The strings VHH, HVH, HHV each give 4 product terms across the row cut, and each of them includes the term of HHH. So D_H has $1 + 3 \cdot 3 = 10$ distinct product terms instead of 13, and D_V has 10 across the column cut; together 20 instead of the $4^3 = 64$ of the crossing.

Lemma 5. *Let $t \geq 1$ and $d(t) = t - 1 + \lceil 8\lfloor t/2 \rfloor/5 \rceil$. Then $5d(t) \leq 9t - 1$, so $d(t) < 1.8t$.*

1. *$D_H = \sum_\xi c_\xi\, g_\xi(x,y)\, h_\xi(u,v)$, with an index ξ of $d(t)$ bits, where g_ξ reads only the top ends x, y and h_ξ only the bottom ends u, v.*
2. *$D_V = \sum_\xi c'_\xi\, g'_\xi(x,u)\, h'_\xi(y,v)$, with an index ξ of $d(t)$ bits, where g'_ξ reads only the left ends and h'_ξ only the right ends.*
3. *D_H is also a sum over an index of $2t$ bits of products across the column cut, and D_V across the row cut.*

The coefficients are dyadic with $O(t)$ bits, and the coefficients and the functions are computed in time polynomial in t.

Proof. The bound. With $k = \lfloor t/2 \rfloor$, $\lceil 8k/5 \rceil \leq (8k+4)/5 \leq (4t+4)/5$, so $5d(t) \leq 5t - 5 + 4t + 4$.

Words. By (3), each V_j is a sum of four products across the row cut, with the letters P, Q, R, H, the top parts $x_j y_j$, 1, $x_j + y_j$, $x_j - y_j$, the bottom parts 1, $u_j v_j$, $u_j + v_j$, $u_j - v_j$, and the coefficients 1, 1, $-\frac{1}{2}$, $\frac{1}{2}$. Each $-H_j$ is the letter H with the coefficient -1. A *word* is $\omega \in \{P, Q, R, H\}^t$; its top part $g_\omega(x,y)$ and bottom part $h_\omega(u,v)$ are the products of the parts of its letters. Multiplying out, T_σ is the sum of the products $g_\omega h_\omega$ over the words ω that *fit* σ, that is, with $\omega_j = H$ wherever $\sigma_j = H$, each with the product of the coefficients of its letters. So $D_H = \sum_\omega c_\omega g_\omega h_\omega$, where

$$c_\omega = \sum_{\substack{\sigma \in \mathcal{H} \\ \omega \text{ fits } \sigma}} \prod_{j:\omega_j \neq H} \kappa(\omega_j) \prod_{j:\omega_j = H} \begin{cases} \frac{1}{2} & \sigma_j = V \\ -1 & \sigma_j = H \end{cases}, \qquad \kappa(P) = \kappa(Q) = 1,\ \kappa(R) = -\tfrac{1}{2}.$$

Which words occur. A string that ω fits has the letter V at each position where $\omega_j \neq H$. Let $\sigma(\omega)$ be the string with V exactly at these positions. Every string that ω fits arises from $\sigma(\omega)$ by changing some letters H into V, so it is H-dominant only if $\sigma(\omega)$ is, because changing V into H keeps H-dominance. So $c_\omega = 0$ unless $\sigma(\omega) \in \mathcal{H}$. The words with $\sigma(\omega) \in \mathcal{H}$ correspond to the pairs of a string $\sigma \in \mathcal{H}$ and a letter from $\{P, Q, R\}$ at each of its positions with V. There are at most $|\mathcal{H}|\, 3^{\lfloor t/2 \rfloor} = 2^{t-1} 3^{\lfloor t/2 \rfloor}$ such words.

The index. The index ξ has two parts. The first part has $t - 1$ bits. Of $\sigma(\omega)$ and its complement, exactly one has the last letter H; call it τ, and let the first part be the letters $\tau_1, \ldots, \tau_{t-1}$, with H written as 1 and V as 0. To decode, put the letter H at the end; the result is τ, and $\sigma(\omega)$ is τ if $\tau \in \mathcal{H}$ and $\bar{\tau}$ otherwise, because exactly one of them is H-dominant. So the first part names the strings of $\mathcal{H}$ bijectively. The second part is a number q of $\lceil 8\lfloor t/2 \rfloor/5 \rceil$ bits: the letters of ω at the positions with V, read from the first position on, are the base-3 digits of q from the lowest on, with P, Q, R as $0, 1, 2$. Since $\sigma(\omega)$ has $j \leq \lfloor t/2 \rfloor$ letters V, $q < 3^j \leq 3^{\lfloor t/2 \rfloor} \leq 2^{8\lfloor t/2 \rfloor/5}$, because $3^5 = 243 < 256 = 2^8$. An index whose second part is at least 3^j names no word, and its term is 0. This proves part 1, with $g_\xi = g_\omega$, $h_\xi = h_\omega$ and $c_\xi = c_\omega$.

The coefficients. In the sum for c_ω, a string that ω fits is $\sigma(\omega)$ with a set A of the positions of the letters H of ω changed to V. Whether it is H-dominant depends only on $|A|$ and on whether A contains the last position. So c_ω is $\prod_j \kappa(\omega_j)$ times a sum of at most $2(t+1)$ terms, each a binomial coefficient times $(\frac{1}{2})^{|A|}(-1)^{h-|A|}$, where h is the number of letters H of ω.

Part 2. Across the column cut, $V_j = (u_j - x_j)(v_j - y_j)$ is one product, of a function of the left ends x_j, u_j and a function of the right ends y_j, v_j, and

$$-H_j = -x_j u_j \cdot 1 - 1 \cdot y_j v_j + \tfrac{1}{2}(x_j + u_j)(y_j + v_j) - \tfrac{1}{2}(u_j - x_j)(v_j - y_j),$$

since $H_j = x_j u_j + y_j v_j - x_j v_j - y_j u_j$. The last term is $-\frac{1}{2}V_j$. So the argument above applies with the letters V and H exchanged: changing H into V keeps V-dominance, a V-dominant string has at most $\lfloor t/2 \rfloor$ letters H, and the first part of the index names τ, the one of the string and its complement with the last letter V, with V written as 1.

Part 3. Across the column cut, V_j is one product and $-H_j$ is four, so every T_σ, and hence D_H, is a sum over the 4^t words with these four letters at each position, with the coefficient of a word summed over the strings of $\mathcal{H}$ that it fits. The index names a word with two bits per letter. The same holds for D_V across the row cut. □

For $t = 3$ the index has $d(3) = 2 + 2 = 4$ bits. The word HQH comes from the string HVH, which has the last letter H, so the first part is $\tau_1, \tau_2 = H, V$, written $1, 0$; the letter Q is the digit 1, so the second part is $0, 1$. For $t = 1$, $\mathcal{H} = \{H\}$ and $D_H = -H_1$ is one product: $d(1) = 0$.

4 One level

Lemma 5 treats one rectangle. A network has wires on many pairs of nodes. We now split all rectangles at once and merge nodes, so that the value becomes a sum of values of networks with fewer nodes.

Wires that carry an index. Lemma 5 writes a piece as $\sum_\xi g_\xi h_\xi$ over an index of m bits. Such a sum is the value of m wires. Let m wires run from a node α to a node β, with tail bits $b \in \{0,1\}^m$ and head bits b'. Put $G(b) = (-1)^{|b|} g_b$ at α, where $|b|$ is the number of ones of b, and $G'(b') = h_{\mathbf{1}-b'}$ at β, where $\mathbf{1} - b'$ complements every bit. The contributing configurations have $b' = \mathbf{1} - b$ and marks with product $(-1)^{|b|}$, so

$$\sum_{b,b'} \prod_{k=1}^{m} (b'_k - b_k)\, G(b)\, G'(b') = \sum_\xi g_\xi h_\xi. \tag{4}$$

The grid. Let N be a network on $K = K'K'$ nodes, written as (a, b) with $a, b \in [K']$: a is the row and b the column. By reversing wires, let every wire run from the node with the smaller number $aK' + b$ to the node with the larger number. A wire between different rows then has its tail in the upper row, and the horizontal pair of Section 2 runs from x to y and from u to v. Two rows $a < a'$ and two columns $b < b'$ form a rectangle ρ. Its *diagonals* are the pairs of nodes $\{(a, b), (a', b')\}$ and $\{(a, b'), (a', b)\}$; every pair of nodes in different rows and different columns is a diagonal of exactly one rectangle. Let w_1 and w_2 be the weights of the two diagonals and $t_\rho = \min(w_1, w_2)$. List the wires of each diagonal in a fixed order, and pair the j-th wire of one diagonal with the j-th wire of the other, for $j \le t_\rho$. These are the *paired* wires; with the ends x_j, y_j at $(a, b), (a, b')$ and u_j, v_j at $(a', b), (a', b')$, the product of their marks is $\prod_j X_j = D^\rho_H + D^\rho_V$, by Section 3. All other wires are *carried*: those inside one row or one column, and the $w_1 + w_2 - 2t_\rho$ unpaired wires of each rectangle.

Choices. A *choice* π picks, for each rectangle ρ with $t_\rho \ge 1$, one of the two pieces $P_\rho \in \{D^\rho_H, D^\rho_V\}$. There are $(K'(K'-1)/2)^2 \le K'^4/4$ rectangles, so at most $2^{K'^4/4}$ choices. In $Z(N)$ the marks of the paired wires multiply to $\prod_\rho (D^\rho_H + D^\rho_V)$. Multiplying out gives $Z(N) = \sum_\pi Z_\pi$, where Z_π is $Z(N)$ with the product of the marks of the paired wires replaced by $\prod_\rho P_\rho$.

Merging the rows. Fix π. For a rectangle ρ with the rows $a < a'$, let d^{r}_ρ be $d(t_\rho)$ if $P_\rho = D^\rho_H$ and $2t_\rho$ if $P_\rho = D^\rho_V$. By Lemma 5, $P_\rho = \sum_\xi g_\xi(x, y) h_\xi(u, v)$ over an index of d^{r}_ρ bits, with the coefficients included in g_ξ. The *row network* N^{r}_π has the K' rows as nodes, and these wires:

- every carried wire between two different rows, with its orientation;
- for every rectangle ρ, d^{r}_ρ new wires from its upper row a to its lower row a', carrying the index of P_ρ as in (4).

The node function at the row a is (1) for the block of the nodes of the row a, with the paired wires treated as follows: their ends at the row a are summed inside the block like the ends of the wires inside the row, their marks are dropped, and the block function has the factor G of (4) for each rectangle with the upper row a and the factor G' for each rectangle with the lower row a. Then the weights of N^{r}_π are

$$w^{\mathrm{r}}(a,a') = \sum_b w((a,b),(a',b)) + \sum_{b<b'} f^{\mathrm{r}}_\rho, \qquad f^{\mathrm{r}}_\rho = w_1 + w_2 - 2t_\rho + d^{\mathrm{r}}_\rho, \tag{5}$$

where ρ is the rectangle with the rows a, a' and the columns b, b', and w_1, w_2 are the weights of its diagonals: f^{r}_ρ counts its unpaired wires and its new wires.

Merging the columns. Exchanging rows and columns gives the *column network* N^{c}_π on the K' columns: the pieces are written across the column cut, d^{c}_ρ is $d(t_\rho)$ if $P_\rho = D^\rho_V$ and $2t_\rho$ if $P_\rho = D^\rho_H$, and $f^{\mathrm{c}}_\rho = w_1 + w_2 - 2t_\rho + d^{\mathrm{c}}_\rho$. The new wires of a rectangle run from its left column to its right column. A carried wire from the node (a,b') to the node (a',b), with $a<a'$ and $b<b'$, now runs from the column b' to the column b; it is reversed, and the node function of the column b has the factor -1 for it.

Lemma 6. *For each choice π, $Z_\pi = Z(N^{\mathrm{r}}_\pi) = Z(N^{\mathrm{c}}_\pi)$. So for each π we may pick the row network or the column network, and $Z(N)$ is the sum over π of the values of the picked networks.*

Proof. Consider the rows. In Z_π, write each P_ρ as the sum over its index and then as the sum (4) over the bits of d^{r}_ρ new wires from the row a to the row a'. The result is the value of a network on the K old nodes: the carried wires, the new wires, and the ends of the paired wires, which are now read by the factors G and G' instead of by marks. The factor G of ρ reads only ends at the row a, at its nodes (a,b) and (a,b'), and G' only ends at the row a'. So merging each row into one node, as in (1), keeps the value and gives N^{r}_π. The columns are the same with the expansion across the column cut in Lemma 5. □

In a rectangle, one of $d^{\mathrm{r}}_\rho, d^{\mathrm{c}}_\rho$ is $d(t_\rho)$ and the other is $2t_\rho$. Let $w_1 \ge w_2$, so $t_\rho = w_2$. In the direction with $d(t_\rho)$, Lemma 5 gives

$$5f_\rho = 5(w_1 - w_2) + 5d(w_2) \le 5w_1 + 4w_2 - 1 \quad (w_2 \ge 1), \tag{6}$$

and $f_\rho = w_1$ if $w_2 = 0$. In the other direction $f_\rho = w_1 + w_2$.

Example. Let four nodes form one rectangle, with three wires between every two of them. A row network keeps the $3+3$ vertical wires and replaces the $3+3$ diagonal wires by $d(3) = 4$ new wires if $P_\rho = D_H$: 10 wires between its two nodes instead of the 12 that cross the row cut. The three wires inside each row are summed inside the new nodes.

5 The potential

A rectangle with D_H has fewer new wires in the row network, and a rectangle with D_V in the column network. A choice may mix them, and a rectangle with wires on only one diagonal keeps all of them in both networks. So the number of wires need not fall. What does change is the

number of nodes. In the network with two rows, two columns and d wires between the top left and the bottom right only, the row network has d wires between its two nodes. Before, these wires occupied one pair of nodes out of six; now they occupy the only pair. So we measure a network N with M nodes and the weights w by the *potential*

$$\Phi(N) = M\Big(\sum_{s,t} w(s,t)^2\Big)^{1/2},$$

where s, t run over the ordered pairs of nodes. In the example, Φ falls from $4\sqrt{2d^2}$ to $2\sqrt{2d^2}$, while the number of wires stays d. We use two consequences of Cauchy. Over the M^2 ordered pairs,

$$2|E| = \sum_{s,t} w(s,t) \le \Phi(N), \tag{7}$$

so a small potential means few wires. For a set R of r nodes, the number of ends at R is $\sum_{s\in R}\sum_t w(s,t)$, a sum over rM ordered pairs, so

$$\sum_{s\in R}\sum_{t} w(s,t) \le \sqrt{rM}\Big(\sum_{s,t} w(s,t)^2\Big)^{1/2} = \sqrt{r/M}\;\Phi(N). \tag{8}$$

Lemma 7. *Let $\gamma = \sqrt{0.91}$. For every choice π, $\min\big(\Phi(N^{\mathrm{r}}_\pi), \Phi(N^{\mathrm{c}}_\pi)\big) \le \gamma\,\Phi(N)$.*

Proof. Fix rows $a \ne a'$, and write each f^{r}_ρ in (5) as $\frac12 f^{\mathrm{r}}_\rho + \frac12 f^{\mathrm{r}}_\rho$. Then (5) has $K' + K'(K'-1) = K'^2$ terms, and Cauchy gives

$$w^{\mathrm{r}}(a,a')^2 \le K'^2\Big(\sum_b w((a,b),(a',b))^2 + \sum_{b<b'} \tfrac12 (f^{\mathrm{r}}_\rho)^2\Big).$$

Sum over the ordered pairs $a \ne a'$ and multiply by K'^2; since N^{r}_π has K' nodes, the left side becomes $\Phi(N^{\mathrm{r}}_\pi)^2$. A rectangle arises from (a,a') and from (a',a), so the $\frac12$ disappears, and the first sum becomes the sum over the ordered pairs of different nodes in one column. The same holds for the columns. So

$$\Phi(N^{\mathrm{r}}_\pi)^2 + \Phi(N^{\mathrm{c}}_\pi)^2 \le K'^4\Big(\sum_\rho \big((f^{\mathrm{r}}_\rho)^2 + (f^{\mathrm{c}}_\rho)^2\big) + \sum_{\substack{s\ne t \text{ in one row}\\ \text{or one column}}} w(s,t)^2\Big).$$

Fix ρ, with $w_1 \ge w_2$. By (6), one of $f^{\mathrm{r}}_\rho, f^{\mathrm{c}}_\rho$ is at most $w_1 + 0.8w_2$, and the other is at most $w_1 + w_2$. Now $(w_1+w_2)^2 \le 2(w_1^2+w_2^2)$, and $(w_1 + 0.8w_2)^2 \le 1.64(w_1^2+w_2^2)$, because the difference is $(0.8w_1 - w_2)^2 \ge 0$. So

$$(f^{\mathrm{r}}_\rho)^2 + (f^{\mathrm{c}}_\rho)^2 \le 3.64\,(w_1^2 + w_2^2).$$

The four ordered pairs on the diagonals of ρ have $\sum w(s,t)^2 = 2(w_1^2+w_2^2)$, so the term of ρ is at most 1.82 times their sum. A pair in one row or one column enters with the factor $1 \le 1.82$. Every ordered pair of different nodes lies in one row, in one column, or on a diagonal of exactly one rectangle. So the right side is at most $1.82\,K'^4 \sum_{s,t} w(s,t)^2 = 1.82\,\Phi(N)^2$, because N has K'^2 nodes. The smaller of the two potentials has a square at most half of this. □

Without the saving of Lemma 5, both f would be $w_1 + w_2$, the constant 3.64 would be 4, and the potential would not fall. The same computation with all $f_\rho = w_1 + w_2$ shows that merging rows or columns never raises the potential.

6 The chain of levels, and its cost

We now iterate until two nodes remain. Let $\ell \geq 1$ and $K_j = 2^{2^{\ell-j}}$ for $0 \leq j \leq \ell$, so that $K_0 = 2^{2^\ell}$, $K_\ell = 2$ and $K_j = K_{j+1}K_{j+1}$.

Theorem 8. *Let N_0 be a network on K_0 nodes with potential Φ_0, whose node functions are evaluated in time T_0. Then $Z(N_0)$ is computed in time $2^{K_0^2/3+3.4\gamma^\ell\Phi_0}$ times T_0+1, times a constant depending on ℓ, times a polynomial in Φ_0 and the bit length of the values of the node functions.*

Proof. The chains. Write the nodes of N_0 as a grid of side K_1. Given N_j with $j < \ell$, a grid of side K_{j+1}, pick a choice π_j for N_j, and let N_{j+1} be the one of its two networks with the smaller potential, and the row network if they are equal; write its K_{j+1} nodes as a grid of side K_{j+2}, and orient its wires as in Section 4. A *chain* is a sequence $(\pi_0, \dots, \pi_{\ell-1})$. By Lemma 6, $Z(N_0)$ is the sum of $Z(N_\ell)$ over all chains. By Lemma 7, $\Phi_j = \Phi(N_j) \leq \gamma^j\Phi_0$.

The network N_j has at most $K_{j+1}^4/4 = K_j^2/4$ rectangles, and N_{j+1} has at most $K_{j+1}^2/4 = K_j/4$, which is at most a quarter of $K_j^2/4$, since $K_j \geq 4$ for $j < \ell$. So all N_j with $j < \ell$ have at most $K_0^2/3$ rectangles together, and there are at most $2^{K_0^2/3}$ chains. For one chain, the weights of all levels are computed in polynomial time, and they fix which of the two networks is kept, since a comparison of potentials is a comparison of two sums of squares.

One chain. The last network N_ℓ has two nodes, and by (7) at most $\Phi_\ell/2 \leq \frac{1}{2}\gamma^\ell\Phi_0$ wires; its value is a sum over at most $2^{\gamma^\ell\Phi_0/2}$ configurations, each with two evaluations of node functions of N_ℓ. A node function of N_{j+1} is (1) for one row or one column of N_j: K_{j+1} nodes of the K_j. By (8) it is a sum over at most 2^{b_j} configurations of ends, with $b_j = \Phi_j/\sqrt{K_{j+1}}$, and each term has K_{j+1} evaluations of node functions of N_j and the factors G, G' of Lemma 5, which take polynomial time. So if T_j is the time of one evaluation at the level j, then $T_{j+1} \leq 2^{b_j}(K_{j+1}T_j + p)$ for a polynomial p, and

$$T_\ell \leq 2^{\sum_{j<\ell} b_j} \prod_{j<\ell} K_{j+1}\,(T_0 + \ell p).$$

The product of the K_{j+1} is a constant depending on ℓ.

The exponent. Put $r = \ell - j \geq 1$. Then $\sqrt{K_{j+1}} = 2^{2^{r-2}} \geq 2^{r/2}$, since $2^{r-2} \geq r/2$ for $r \geq 1$. With $\Phi_j \leq \gamma^\ell\Phi_0\gamma^{-r}$,

$$\sum_{j<\ell} b_j \leq \gamma^\ell\Phi_0 \sum_{r\geq 1} q^r = \gamma^\ell\Phi_0 \frac{q}{1-q}, \qquad q = \frac{1}{\gamma\sqrt{2}} = \frac{1}{\sqrt{1.82}} < 0.7413,$$

and $q/(1-q) < 2.87$. Adding the $\frac{1}{2}\gamma^\ell\Phi_0$ of the last level gives at most $3.4\gamma^\ell\Phi_0$. From one level to the level before it, the bound on Φ_j grows by the factor $1/\gamma$, but the share $1/\sqrt{K_{j+1}}$ of one row is the square of the next one, so it falls by the factor $1/\sqrt{2}$ or more; so the bounds on the b_j fall by the factor q towards the first level.

Bits. All numbers of wires are at most the number of wires of N_0, since $d(t) \leq 2t$, and so at most Φ_0. If the values at the level j have at most B_j bits, a value at the level $j+1$ is a sum of at most 2^{b_j} products of K_{j+1} values and of the coefficients of Lemma 5, so $B_{j+1} \leq b_j + K_{j+1}B_j + O(\Phi_0 + K_0^2)$. Hence all values are dyadic with polynomially many bits, and the arithmetic adds a polynomial factor. □

For comparison, enumerating the ends of all wires costs $2^{|E|}$, and $|E|$ can be close to $\Phi_0/2$. The theorem replaces this exponent by $3.4\gamma^\ell\Phi_0$, at the cost of a factor $2^{K_0^2/3}$ that does not depend on the wires.

7 Placing a formula, and the end of the proof

Theorem 8 helps when $\Phi_0 = O(n)$ and the first node functions are cheap. Both hold when the formula has few occurrences and they are spread evenly over the nodes.

Lemma 9. *Let $K \geq 2$, $\Delta \geq 1$, and $n \geq K^2$, and put $D = \Delta + 1$. Let Ψ be a formula as in Theorem 1. In polynomial time, the nodes of the network of Ψ can be partitioned into K blocks such that each block has at most $2Dn/K$ wires inside it, and the merged network N_0, with its wires between blocks, has $\Phi(N_0) \leq 6Dn$.*

Proof. Every wire has its tail at a variable, and a variable has at most Δ occurrences and its own wire, so at most D wires.

Variables. Put the variable i into the block $i \bmod K$. A block holds at most $\lceil n/K \rceil \leq n/K + 1 \leq 2n/K$ variables, since $n \geq K$, so it has at most $2Dn/K$ wires inside it.

Clauses and own nodes. Let $L(a, b)$ count the wires placed so far whose tail (a variable) lies in the block a and whose head (a clause or an own node) lies in the block b. For each a, $\sum_b L(a, b) \leq D(n/K + 1)$ at all times, whatever the placement of the other nodes, because the block a has at most $n/K + 1$ variables with at most D wires each. Go through the clauses and the own nodes in any order. Such a node has at most three wires; let a_1, a_2, a_3 be the blocks of their tails, with repetitions. Put the node into the block b that minimizes $\sum_k L(a_k, b)$. Summed over all K blocks b, this quantity is at most $3D(n/K + 1)$, so its minimum is at most $3D(n/K + 1)/K$. Placing the node raises $L(a, b)$ only for $a \in \{a_1, a_2, a_3\}$ and the chosen b, by at most 3, and before that raise $L(a, b)$ was at most the minimum. So at the end every $L(a, b) \leq 3D(n/K + 1)/K + 3$.

Potential. For $a \neq b$, $w(a, b) = L(a, b) + L(b, a) \leq 6D(n/K + 1)/K + 6$, and $\sum_{a,b} w(a, b)$ is twice the number of wires between blocks, at most $2Dn$. So

$$\Phi(N_0)^2 = K^2 \sum_{a,b} w(a, b)^2 \leq K^2 \Big(\frac{6D(n + K)}{K^2} + 6 \Big) 2Dn = 12D^2n^2 + 12D^2nK + 12DK^2n \leq 36D^2n^2,$$

since $K \leq K^2 \leq n$ and $D \geq 1$. □

Proof of Theorem 1. If Ψ has an empty clause, the answer is 0. Put $D = \Delta + 1$. Let $\ell \geq 1$ be the least integer such that $20.4\, D\gamma^\ell + 2D/K \leq \varepsilon$ for $K = 2^{2^\ell}$; it exists because the left side tends to 0. The parameters ℓ and K depend only on Δ and ε. If $n < K^2$, enumerate the $2^n < 2^{K^2}$ assignments. Otherwise place the network of Ψ by Lemma 9, and merge each block with equality tests: a wire inside a block becomes one bit summed inside the block function. By Lemma 4, the merged network has $Z_=$ equal to the number of solutions. Orient every wire between blocks from the block with the smaller number to the larger, and apply Lemma 3: the result is a network N_0 with difference marks and the same value. A node function of N_0 is a sum over at most $2^{2Dn/K}$ bits of products of node functions, so it is evaluated in time $2^{2Dn/K}$ times a polynomial. Theorem 8 gives time $2^{K^2/3}$ times a constant, times a polynomial, times

$$2^{3.4\gamma^\ell \cdot 6Dn + 2Dn/K} \leq 2^{\varepsilon n}.$$

The values of the first node functions have $O(n)$ bits, and the formula has at most Δn clauses, so the polynomial is a polynomial in n. □

Proof of Corollary 2. The sparsification lemma [IPZ01] gives, for each $\varepsilon > 0$, a number Δ and an algorithm that maps a 3-CNF φ on n variables to at most $2^{\varepsilon n}$ formulas on the same variables,

with clauses of at most three literals and at most Δ occurrences of each variable, in time $2^{\varepsilon n}$ times a polynomial, such that the solutions of φ are the union of their solutions. Count the solutions of each by Theorem 1 with ε, and accept if one count is positive. The time is $2^{2\varepsilon n}$ times a constant and a polynomial in $|\varphi|$, and $n \le |\varphi|$ since the input declares its variables. $\square$

The rate. The proof fixes ε. If ℓ grows with n, so that $K_0^2 = 2^{2^{\ell+1}}$ is about $n/\log n$, then $\ell \approx \log_2 \log_2 n$ and $\gamma^\ell \approx (\log n)^{-0.068}$, since $\log_2 \gamma \approx -0.068$; the exponent for fixed Δ becomes $O(n/(\log n)^{0.068})$. OpenAI and Claude reach $O(n/\sqrt{\log n})$ for fixed Δ [OAI26, Cla26], which corresponds to a factor $1/\sqrt{2}$ per level. The difference comes from the pieces: in Lemma 7 a rectangle contributes $(f^{\mathrm{r}})^2 + (f^{\mathrm{c}})^2$, and the bound 1.82 becomes 1 when this sum is at most the $2(w_1^2 + w_2^2)$ of the rectangle. Our two pieces cost about $(2t, 1.8t)$ for $t = w_1 = w_2$, while the pieces of [Cla26, Appendix B], more of them per rectangle, have costs close to a circle of radius $2t$.

8 Formalization

The Lean 4 development [MU21, mC20] is the library `DominantStrings` in the directory `lean/` of the repository `https://github.com/suquark/NotETHSimplified`. It uses the libraries `TransposeBase` and `SatBase` of that project for the word language, helpers for brute force, binomials and fields, the network of a formula, and the machine layer, and `FormalizationKit` for resource bounds on the machine. The end statements for Theorem 1 are defined in `DominantStrings/Statements.lean`. The file `DominantStrings/Audit.lean` checks by `rfl` that the end theorems are these statements, and it reports that the theorems listed there depend only on the axioms `propext`, `Classical.choice` and `Quot.sound`, or on a subset of them; the library contains no `sorry`, and its modules were replayed by the kernel checker `leanchecker`. A meta-program, `DominantStrings/ClosureCheck.lean`, computes the closure of `Machine.machine_bounded` and counts no constant with a name containing `sparseTree`, `SparseState`, `chooseSparseRule` or `treeE`, the branching tree of the sparsification, nor any containing `OnePigeonhole`, `Chebyshev`, `Fourier`, `Hermite`, `Matrix.det` or `indow`. Table 1 maps the paper to the Lean declarations.

Paper	Lean declaration	Module
Lemma 3	`val_flip`	`Base`
Lemma 4, merging	`BaseNet.valEq_base`, `BaseNet.val_base`, `cnf_net_main`	`Base`, `TransposeBase`
Lemma 5	`Ppol_add`, `seed_rows`, `seed_cols`, `seed_sum`, `sum_cheap`, `dLen_le`	`Seed`
Lemma 6, (4)	`step_val`, `sum_mk_index`, `sum_pair_prod`	`Step`, `Mark`
Lemma 7	`dLen_cheap_le`, `sq_two_widths`, `potential_two`, `potential_min`	`Seed`, `Potential`
Theorem 8	`chain_val`, `chainVal_eq`, `core_spec`	`Chain`, `Pseudo.CoreSpec`
Lemma 9	`load_table_le`, `psi_table_le_all`, `BaseNet.w0_total`	`Greedy`, `Base`
Theorem 1	`Pseudo.Final.pseudo_bounded`, `Machine.machine_bounded`	`Pseudo.Bounded`, `Machine.Bounded`
Corollary 2	`Pseudo.Final.pseudo_main`, `Machine.not_ETH`	`Pseudo.Final`, `Machine.FinalMain`

Table 1: Map from the paper to the Lean development. Names are in `DominantStrings`, except `cnf_net_main` in `TransposeBase`.

The end statements. The word language has addition, subtraction and multiplication modulo 2^{w}, division, remainder, powers of 2, comparison, and bounded loops; its semantics gives the value of a program and the number of word operations. The theorem `Pseudo.Final.pseudo_bounded` states: for every bound D and every $\varepsilon > 0$ there are one program P and constants C, b, c such that, for every n and every formula F with n variables, clauses of at most three literals and at

most D occurrences of each variable, for every word size $\mathsf{w} \geq c(n+1)^3$ and every environment that continues below the bitset of F and the number n, the value of P is nonzero exactly when F is satisfiable, and the number of word operations is at most $C(n+1)^b 2^{\varepsilon n}$. The program enumerates the assignments for $n \leq K^2$ and runs one chain otherwise; it has no branching. The theorem `Machine.machine_bounded` compiles it: for every D and $\varepsilon > 0$ there are a primitive multitape machine, in the model of OpenAI's Lean development [OAI26b], and constants such that for every formula φ with clauses of at most three literals in which every variable occurs in at most D clauses counted with their sign, the machine started on the fixed encoding of φ halts with the correct answer within $C(|\varphi|+1)^d 2^{\varepsilon n}$ steps, and every prefix of that run visits at most $C'(|\varphi|+1)^{d'}$ tape cells. The library also joins the same chain with the branching tree of `SatBase` into `Machine.not_ETH`, the formal form of Corollary 2; the sparsification lemma is proved there in its own form, and it is not part of this paper.

Departures from the text. Lean follows the text: the value with difference marks (`val`), the crossing as the product of the marks of two bundles (`cross`), the letters P, Q, R and the special letter with their top and bottom functions (`topL`, `botL`), the dominant strings with the tie broken by the last letter (`adm`), the two-part index (`markOf`, `letterCheap`), the index wires with the sign at the tail and the complement at the head (`Lev.segT`), the orientation from the smaller node, and the first level built by Lemmas 4 and 3 (`flipF`). The remaining differences are these. The pieces of a rectangle are multiplied by a fixed power of two, so that all node functions are integers, and the value of a chain is a power of two times the number of solutions (`core_spec`); the end theorems state the decision. A choice is encoded in K'^4 bits with a validity test. The network of a formula is the shared network of `TransposeBase`; its clause functions allow clause sets that hold both signs of a variable and clauses of fewer than three literals, and Lean counts the occurrences of a variable as the clauses with x_i plus the clauses with $\neg x_i$. The constants are Lean's own: the potential of the first level is bounded by $28((D+4)n)^2$ in place of $(6Dn)^2$, and `exists_rate` uses the rate $(477/500)^H$, with $100(477/500)^2 \geq 91$, and the ratio $21/40$ for the sum over the levels.

What is not formalized. Theorem 8 is stated in Lean only as `Pseudo.core_spec`, for placed networks of formulas, in word operations, with its exponent as an unsummed sum; the value part, `chain_val`, holds for arbitrary symmetric weights. The constants 3.4 and $6D$ of this text are computed by hand.

Use of AI

The argument and the Lean proofs were developed in interactive sessions with Claude (Anthropic), building on the public manuscripts [OAI26, O'D26, Cla26, Kin26]. The author is responsible for any errors.

References

[Cla26] Claude (Anthropic). A $2^{o(n)}$ time algorithm for 3-SAT. Preprint, October 26, 2026. https://github.com/anthropics/formal-math/tree/16a5ce41304fcdaea4e0be7c132e1c361dc26da5/eth.

[ETH26] R. Impagliazzo, R. Paturi, and R. Williams. ETH refutation: examination and discussion. Web site, October 2026. https://ethdiscussion.blog/2026/10/26/eth-refutation-examination-and-discussion/.

[IP01] R. Impagliazzo and R. Paturi. On the complexity of k-SAT. *J. Comput. System Sci.* 62(2):367–375, 2001.

[IPZ01] R. Impagliazzo, R. Paturi, and F. Zane. Which problems have strongly exponential complexity? *J. Comput. System Sci.* 63(4):512–530, 2001.

[Kin26] S. Kintali. A simplified proof of ETH is false. Manuscript, October 26, 2026. https://shivakintali.github.io/papers/ETH.pdf.

[MU21] L. de Moura and S. Ullrich. The Lean 4 theorem prover and programming language. In *Proc. 28th CADE*, 2021.

[mC20] The mathlib Community. The Lean mathematical library. In *Proc. 9th ACM SIGPLAN CPP*, 2020.

[OAI26] OpenAI. The Exponential Time Hypothesis is false. Manuscript, October 17, 2026; revised October 25, 2026. https://github.com/openai/math/blob/42907c9e999cc00d6abe7a50bfc1023250edd1b9/preprints/The-Exponential-Time-Hypothesis-is-False-October-25-2026/paper.pdf.

[OAI26b] OpenAI. Lean formalization accompanying "The Exponential Time Hypothesis is false". Repository, commit 4eaee87; see `lean/OAI/Algorithms/SubexponentialSAT/SqrtLogSpace/CompileSpace.lean`. https://github.com/openai/math/tree/4eaee87548250b2682affda7d05d947c05554bae/lean.

[O'D26] R. O'Donnell. Some exposition of "The Exponential Time Hypothesis Is False". Manuscript, October 30, 2026. https://www.cs.cmu.edu/~odonnell/some_eth_false_exposition.pdf.

[Zhu26] S. Zhuang. Uncrossing Wires for 3-SAT: A potential that closes the recursion. Preprint, October 31, 2026. https://github.com/suquark/NotETHSimplified/tree/main/papers/uncrossing-wires.